**ALGORITMOS E PENSAMENTO COMPUTACIONAL**
(ALGPC)

**ATIVIDADE 02**

# CAIXA ELETRÔNICO

*Relatórios, pseudocódigos, códigos em C, testes e feedbacks entre os grupos*

---


**Instituição:** UNICID
**Curso / disciplina:** Algoritmos e Pensamento Computacional
**Professor(a):** Osvaldo Camargo Botelho Dos Santos

**INTEGRANTES**

**GRUPO 1**
Alessandra Guimarães da Silva
Davi Pereira de Oliveira Silva
Ellen Cristina Soares de Jesus
Samuel Guimarães da Silva

**GRUPO 2**
Gabriel Anjos Albuquerque
João Pedro Theodoro de Sousa Andrade
Samuel Carmoni de Oliveira
Petterson Michel Lima Aguilar


**2026**

PARTE 1

# Atividade 02 – Caixa Eletrônico – Grupo 1

Algoritmos e Pensamento Computacional (ALGPC) · Integrantes:
Alessandra Guimarães da Silva
Davi Pereira de Oliveira Silva
Ellen Cristina Soares de Jesus
Samuel Guimarães da Silva

> **Observação:** além do que o enunciado pede (o saque), o programa inclui acréscimos feitos por iniciativa própria, a partir do que foi conversado em sala: 3 clientes com senha (3 tentativas e bloqueio do cartão), menu com ver saldo, depósito e extrato, e um estoque fixo de notas no caixa. Esses itens não fazem parte do enunciado da atividade.

## Passo 1: Entenda o problema

### O que é um caixa eletrônico simplificado?

É um programa que simula o saque de um caixa eletrônico (ATM): a pessoa informa quanto quer sacar, o programa confere se o valor pode ser sacado e, se puder, mostra quantas notas de cada valor (100, 50, 20, 10 e 5) serão entregues. Ele é simplificado porque faz só o essencial: trabalha apenas com valores inteiros e com essas cinco notas (sem moedas nem centavos) e guarda os dados em variáveis do próprio programa, sem ligação com um banco de verdade. Além do saque, o programa tem os acréscimos descritos na Observação do início do documento.

### Quais são as entradas e saídas do sistema?

Entradas:

- valor do saque (valorSaque): número inteiro digitado pela pessoa;
- saldo disponível (saldo): dado do sistema; nos testes, R$ 500;
- nos acréscimos do programa: cliente (1 a 3), senha, opção do menu e valor do depósito.

Saídas:

- quantidade de notas de cada valor que serão entregues (só as que forem maiores que zero) e o novo saldo;
- mensagem de erro quando o saque não pode ser feito (valor inválido, saldo insuficiente ou caixa sem notas suficientes);
- nos acréscimos: saldo, extrato, avisos de senha incorreta e de cartão bloqueado.

### Quais validações são necessárias?

No saque, o programa confere, nesta ordem:

- se o valor é maior que zero (recusa o 0 e os números negativos);
- se o valor é múltiplo de 5, porque a menor nota é a de 5 (valorSaque % 5 tem que dar 0);
- se o valor é menor ou igual ao saldo;
- depois de calcular as notas, se ainda sobrar valor sem nota para pagar (restante diferente de 0), o caixa não consegue entregar e o saque é recusado.

Para entrar no sistema, as senhas dos 3 clientes são: cliente 1 (1234), cliente 2 (2222) e cliente 3 (3333).

Nos acréscimos, o programa também confere se o cliente é de 1 a 3, se a senha está correta (3 tentativas; depois disso o cartão é bloqueado) e se o depósito é maior que zero e múltiplo de 5.

### Como distribuir as notas de forma eficiente?

Começando sempre pela nota de maior valor (100) e descendo até a de 5. Assim, havendo notas suficientes no caixa, o saque usa a menor quantidade possível de notas. Para cada nota, a divisão inteira (/) diz quantas notas cabem no valor e o resto (%) diz quanto ainda falta pagar, que passa para a nota seguinte. Exemplo com o saque de 285:

```
285 / 100 = 2 notas de 100   (285 % 100 = 85)
 85 /  50 = 1 nota de 50    ( 85 %  50 = 35)
 35 /  20 = 1 nota de 20    ( 35 %  20 = 15)
 15 /  10 = 1 nota de 10    ( 15 %  10 =  5)
  5 /   5 = 1 nota de 5     (  5 %   5 =  0)
```

Como o caixa do programa tem estoque limitado, se faltar nota de algum valor ele entrega só as que existem e passa o que sobrou para as notas menores; se no fim ainda sobrar valor, o saque é recusado.

## Passo 2: Escreva o pseudocódigo

O programa tem um loop central que atende os 3 clientes e só termina quando todos saem (ou são bloqueados) ou quando as notas do caixa acabam. O saque é a parte principal da atividade (valor, validações e distribuição das notas).

```
INICIO
    // Declaracao de variaveis do saque
    INTEIRO valorSaque, restante
    INTEIRO notas100, notas50, notas20, notas10, notas5

    // Declaracao de variaveis do caixa e dos clientes
    INTEIRO estoque100, estoque50, estoque20, estoque10, estoque5, totalNotas
    INTEIRO saldo[3], senha[3], tentativas[3], qtdTransacoes[3], historico[3][10]
    INTEIRO opcao_cliente, c, opcao, senhaDigitada, valorDeposito, i
    LOGICO bloqueado[3], finalizou[3], logado, sair, todosFinalizados

    // Operadores: DIV = divisao inteira (/ em C) e MOD = resto da divisao (% em C)

    // Valores iniciais
    senha <- {1234, 2222, 3333}
    saldo <- {500, 1000, 300}
    tentativas <- {0, 0, 0}
    qtdTransacoes <- {0, 0, 0}
    bloqueado <- {FALSO, FALSO, FALSO}
    finalizou <- {FALSO, FALSO, FALSO}
    estoque100 <- 10
    estoque50 <- 10
    estoque20 <- 20
    estoque10 <- 20
    estoque5 <- 20

    // Loop central: so termina quando todos finalizam ou as notas acabam
    ENQUANTO VERDADEIRO FACA
        totalNotas <- estoque100 + estoque50 + estoque20 + estoque10 + estoque5

        todosFinalizados <- VERDADEIRO
        PARA i DE 0 ATE 2 FACA
            SE finalizou[i] = FALSO E bloqueado[i] = FALSO ENTAO
                todosFinalizados <- FALSO
            FIMSE
        FIMPARA

        SE totalNotas = 0 ENTAO
            ESCREVA "O caixa ficou sem notas. Encerrando."
            PARE
        FIMSE
        SE todosFinalizados = VERDADEIRO ENTAO
```

```
            ESCREVA "Todos os clientes ja foram atendidos. Encerrando."
            PARE
        FIMSE

        // Escolha do cliente
        ESCREVA "Informe qual cliente e: 1, 2 ou 3"
        LEIA opcao_cliente
        SE opcao_cliente < 1 OU opcao_cliente > 3 ENTAO
            ESCREVA "Cliente invalido!"
            VOLTE AO INICIO DO LOOP
        FIMSE
        c <- opcao_cliente - 1
        SE finalizou[c] = VERDADEIRO ENTAO
            ESCREVA "Este cliente ja finalizou o atendimento."
            VOLTE AO INICIO DO LOOP
        FIMSE
        SE bloqueado[c] = VERDADEIRO ENTAO
            ESCREVA "Cartao bloqueado!"
            VOLTE AO INICIO DO LOOP
        FIMSE

        // Login: 3 tentativas por cliente
        logado <- FALSO
        ENQUANTO tentativas[c] < 3 E logado = FALSO FACA
            ESCREVA "Digite sua senha: "
            LEIA senhaDigitada
            SE senhaDigitada = senha[c] ENTAO
                logado <- VERDADEIRO
                tentativas[c] <- 0
            SENAO
                tentativas[c] <- tentativas[c] + 1
                SE tentativas[c] < 3 ENTAO
                    ESCREVA "Senha incorreta! Tentativas restantes: ", 3 - tentativas[c]
                FIMSE
            FIMSE
        FIMENQUANTO
        SE logado = FALSO ENTAO
            bloqueado[c] <- VERDADEIRO
            ESCREVA "Senha errada 3 vezes. Cartao bloqueado!"
            VOLTE AO INICIO DO LOOP
        FIMSE
        ESCREVA "Bem-vindo, cliente ", opcao_cliente

        // Menu do cliente: repete ate escolher sair
        sair <- FALSO
        ENQUANTO sair = FALSO FACA
            ESCREVA "1 - Ver saldo | 2 - Depositar | 3 - Sacar | 4 - Extrato | 0 - Sair"
            LEIA opcao

            ESCOLHA opcao

                CASO 1: // Ver saldo
                    ESCREVA "Seu saldo e: ", saldo[c]

                CASO 2: // Depositar
                    ESCREVA "Digite o valor do deposito (multiplo de 5):"
                    LEIA valorDeposito
                    SE valorDeposito <= 0 ENTAO
                        ESCREVA "Digite um valor maior que zero."
                    SENAO SE valorDeposito MOD 5 <> 0 ENTAO
                        ESCREVA "So aceitamos notas de 100, 50, 20, 10 e 5. Digite um multiplo de 5."
                    SENAO
                        saldo[c] <- saldo[c] + valorDeposito
                        // extrato cheio: descarta a mais antiga
                        SE qtdTransacoes[c] = 10 ENTAO
                            PARA i DE 0 ATE 8 FACA
                                historico[c][i] <- historico[c][i + 1]
                            FIMPARA
                            qtdTransacoes[c] <- 9
                        FIMSE
                        historico[c][qtdTransacoes[c]] <- valorDeposito
                        qtdTransacoes[c] <- qtdTransacoes[c] + 1
                        ESCREVA "Deposito realizado! Novo saldo: ", saldo[c]
                    FIMSE

                CASO 3: // SAQUE
```

```
                // 1) Solicita o valor
                ESCREVA "Digite o valor do saque: "
                LEIA valorSaque

                // 2) Valida: positivo, multiplo de 5 e dentro do saldo
                SE valorSaque <= 0 ENTAO
                    ESCREVA "Valor invalido! Digite um valor maior que zero."
                SENAO SE valorSaque MOD 5 <> 0 ENTAO
                    ESCREVA "Valor invalido! O saque deve ser multiplo de 5 (notas de 100, 50, 20,
    10 e 5)."
                SENAO SE valorSaque > saldo[c] ENTAO
                    ESCREVA "Saldo insuficiente! Seu saldo e R$ ", saldo[c]
                SENAO
                    // 3) Calcula as notas, comecando pela maior
                    // DIV da a quantidade de notas e MOD da o que ainda falta pagar
                    restante <- valorSaque

                    // notas de 100
                    notas100 <- restante DIV 100
                    SE notas100 > estoque100 ENTAO
                        notas100 <- estoque100
                        restante <- restante - notas100 * 100
                    SENAO
                        restante <- restante MOD 100
                    FIMSE

                    // notas de 50
                    notas50 <- restante DIV 50
                    SE notas50 > estoque50 ENTAO
                        notas50 <- estoque50
                        restante <- restante - notas50 * 50
                    SENAO
                        restante <- restante MOD 50
                    FIMSE

                    // notas de 20
                    notas20 <- restante DIV 20
                    SE notas20 > estoque20 ENTAO
                        notas20 <- estoque20
                        restante <- restante - notas20 * 20
                    SENAO
                        restante <- restante MOD 20
                    FIMSE

                    // notas de 10
                    notas10 <- restante DIV 10
                    SE notas10 > estoque10 ENTAO
                        notas10 <- estoque10
                        restante <- restante - notas10 * 10
                    SENAO
                        restante <- restante MOD 10
                    FIMSE

                    // notas de 5
                    notas5 <- restante DIV 5
                    SE notas5 > estoque5 ENTAO
                        notas5 <- estoque5
                        restante <- restante - notas5 * 5
                    SENAO
                        restante <- restante MOD 5
                    FIMSE

                    // 4) Mostra o resultado ou a mensagem de erro
                    SE restante <> 0 ENTAO
                        ESCREVA "O caixa nao tem notas suficientes para esse valor. Tente outro
    valor."
                    SENAO
                        estoque100 <- estoque100 - notas100
                        estoque50 <- estoque50 - notas50
                        estoque20 <- estoque20 - notas20
                        estoque10 <- estoque10 - notas10
                        estoque5 <- estoque5 - notas5
                        saldo[c] <- saldo[c] - valorSaque
                        // extrato cheio: descarta a mais antiga
                        SE qtdTransacoes[c] = 10 ENTAO
                            PARA i DE 0 ATE 8 FACA
```

```
                                historico[c][i] <- historico[c][i + 1]
                            FIMPARA
                            qtdTransacoes[c] <- 9
                        FIMSE
                        historico[c][qtdTransacoes[c]] <- -valorSaque
                        qtdTransacoes[c] <- qtdTransacoes[c] + 1

                        ESCREVA "Notas entregues:"
                        SE notas100 > 0 ENTAO ESCREVA notas100, " nota(s) de 100" FIMSE
                        SE notas50 > 0 ENTAO ESCREVA notas50, " nota(s) de 50" FIMSE
                        SE notas20 > 0 ENTAO ESCREVA notas20, " nota(s) de 20" FIMSE
                        SE notas10 > 0 ENTAO ESCREVA notas10, " nota(s) de 10" FIMSE
                        SE notas5 > 0 ENTAO ESCREVA notas5, " nota(s) de 5" FIMSE
                        ESCREVA "Novo saldo: ", saldo[c]

                        totalNotas <- estoque100 + estoque50 + estoque20 + estoque10 + estoque5
                        SE totalNotas = 0 ENTAO
                            ESCREVA "O caixa ficou sem notas! Voce ainda pode ver saldo e extrato."
                        FIMSE
                    FIMSE
                FIMSE

            CASO 4: // Extrato
                SE qtdTransacoes[c] = 0 ENTAO
                    ESCREVA "Nenhuma transacao realizada."
                SENAO
                    PARA i DE 0 ATE qtdTransacoes[c] - 1 FACA
                        SE historico[c][i] > 0 ENTAO
                            ESCREVA i + 1, ") Deposito: ", historico[c][i]
                        SENAO
                            ESCREVA i + 1, ") Saque: ", -historico[c][i]
                        FIMSE
                    FIMPARA
                    ESCREVA "Obs.: o extrato mostra apenas as ultimas 10 transacoes."
                FIMSE

            CASO 0: // Sair
                ESCREVA "Obrigado e ate logo, cliente ", opcao_cliente
                sair <- VERDADEIRO

            OUTRO CASO:
                ESCREVA "Opcao invalida!"

        FIMESCOLHA
    FIMENQUANTO

    finalizou[c] <- VERDADEIRO
FIMENQUANTO
FIM
```

## Passo 3: Identifique as variáveis necessárias

Variáveis pedidas no enunciado:

| Variável | Tipo | Finalidade |
|---|---|---|
| `valorSaque` | int | Valor que o usuário deseja sacar |
| `saldo` | int (vetor de 3) | Saldo disponível na conta de cada cliente |
| `notas100, notas50, notas20, notas10, notas5` | int | Contadores: quantas notas de cada tipo serão entregues no saque |
| `restante` | int | Valor que ainda falta pagar após cada cálculo |

Variáveis extras (para os 3 clientes e o estoque de notas do caixa):

| Variável | Tipo | Finalidade |
|---|---|---|
| `estoque100, estoque50, estoque20, estoque10, estoque5` | int | Quantidade de notas que existem no caixa (diminui a cada saque) |
| `totalNotas` | int | Soma do estoque; quando chega a 0 o caixa acabou |
| `senha` | int (vetor de 3) | Senha de cada cliente |
| `tentativas` | int (vetor de 3) | Quantidade de senhas erradas de cada cliente |
| `bloqueado` | bool (vetor de 3) | Indica se o cartão do cliente foi bloqueado |
| `finalizou` | bool (vetor de 3) | Indica se o cliente já terminou o atendimento |
| `historico, qtdTransacoes` | int (matriz 3x10 e vetor de 3) | Extrato de cada cliente, com as últimas 10 transações (depósito positivo, saque negativo) |
| `opcao_cliente, c` | int | Cliente escolhido (1 a 3) e sua posição nos vetores (0 a 2) |
| `opcao` | int | Opção escolhida no menu |
| `senhaDigitada, valorDeposito, i` | int | Senha lida, valor do depósito e contador de laços |
| `logado, sair, todosFinalizados` | bool | Controle do login, do menu e do loop central |

## Passo 4: Implemente a lógica em C

O código-fonte está no arquivo `caixa_eletronico_atividade2.c` e segue o pseudocódigo do Passo 2. No saque, ele valida o valor (maior que zero, múltiplo de 5 e dentro do saldo) e calcula as notas começando pela de 100: `/` dá a quantidade de notas e `%` dá o que ainda falta pagar. Se o caixa tiver menos notas de um tipo do que o necessário, entrega só as que existem e passa o que sobrou para as notas menores; se no fim ainda sobrar valor sem nota, o saque é recusado. As mensagens do programa foram escritas sem acento de propósito, porque o terminal do Windows pode exibir letras acentuadas de forma errada (um comentário no início do código explica isso). O extrato de cada cliente guarda as últimas 10 transações e avisa isso quando é exibido.

## Passo 5: Teste seu programa

Testamos os valores sugeridos no enunciado (50, 95, 123 e 1005) e também o 0 e o -10, que são casos extremos, sempre com o cliente 1, que começa com saldo de R$ 500. Cada valor foi uma

execução nova do programa. As saídas abaixo são trechos da execução (o login e o menu foram omitidos) e as mensagens aparecem sem acento, como explicado no Passo 4.

### Teste 1: saque de 50 (válido)

**Entrada:** saque de R$ 50, com saldo inicial de R$ 500.

```
Digite o valor do saque: 50

Notas entregues:
1 nota(s) de 50

Novo saldo: R$ 450
```

Correto: 50 é múltiplo de 5, cabe no saldo e é pago com 1 nota de 50.

### Teste 2: saque de 95 (válido)

**Entrada:** saque de R$ 95, com saldo inicial de R$ 500.

```
Digite o valor do saque: 95

Notas entregues:
1 nota(s) de 50
2 nota(s) de 20
1 nota(s) de 5

Novo saldo: R$ 405
```

Correto: 50 + 2×20 + 5 = 95, e o saldo passa de R$ 500 para R$ 405.

### Teste 3: saque de 123 (inválido)

**Entrada:** saque de R$ 123, com saldo inicial de R$ 500.

```
Digite o valor do saque: 123
Valor invalido! O saque deve ser multiplo de 5 (notas de 100, 50, 20, 10 e 5).
```

Correto: 123 % 5 = 3, então o valor não é múltiplo de 5 e o saque é recusado sem alterar o saldo.

### Teste 4: saque de 1005 (inválido)

**Entrada:** saque de R$ 1005, com saldo inicial de R$ 500.

```
Digite o valor do saque: 1005
Saldo insuficiente! Seu saldo e R$ 500.
```

Correto: 1005 é múltiplo de 5, mas é maior que o saldo de R$ 500, então o saque é recusado.

### Teste 5: saque de 0 (inválido)

**Entrada:** saque de R$ 0, com saldo inicial de R$ 500.

```
Digite o valor do saque: 0
Valor invalido! Digite um valor maior que zero.
```

Correto: 0 não é maior que zero, então o saque é recusado sem alterar o saldo.

### Teste 6: saque de -10 (inválido)

**Entrada:** saque de -10 (valor negativo), com saldo inicial de R$ 500.

```
Digite o valor do saque: -10
Valor invalido! Digite um valor maior que zero.
```

Correto: -10 é menor que zero, então o saque é recusado sem alterar o saldo. A validação de valor positivo vem antes da de múltiplo de 5, por isso a mensagem é a de valor maior que zero, mesmo o -10 sendo múltiplo de 5.

## Passo 6: Faça ajustes se necessário

Os seis testes do Passo 5 deram o resultado esperado, então o cálculo e as validações do saque não precisaram de correção.

## Exemplo de execução: saque de 285

Trecho da saída real do programa (cliente 1, saldo de R$ 500) para o saque de 285:

```
Digite o valor do saque: 285

Notas entregues:
2 nota(s) de 100
1 nota(s) de 50
1 nota(s) de 20
1 nota(s) de 10
1 nota(s) de 5

Novo saldo: R$ 215
```

O resultado bate com o exemplo do enunciado: 285 = 2×100 + 50 + 20 + 10 + 5.

## Código-fonte do Grupo 1 (caixa_eletronico_atividade2.c)

Listagem completa do arquivo caixa_eletronico_atividade2.c, citado no Passo 4.

```
#include <stdio.h>
#include <stdbool.h>

/*
 * Atividade 02 - Caixa Eletrônico
 * Algoritmos e Pensamento Computacional (ALGPC)
 * Integrantes: Alessandra Guimarães da Silva, Davi Pereira de Oliveira Silva,
 * Ellen Cristina Soares de Jesus e Samuel Guimarães da Silva
 *
 * Caixa eletrônico com 3 clientes e quantidade fixa de notas (100, 50, 20, 10 e 5).
 * O caixa dá sempre as maiores notas possíveis primeiro.
 *
 * OBSERVAÇÃO SOBRE OS ACENTOS: as mensagens que o programa mostra na tela estão
 * escritas SEM acento de propósito, porque o terminal do Windows pode exibir
 * letras acentuadas de forma errada (símbolos estranhos no lugar das letras).
 *
 * OBSERVAÇÃO SOBRE O ESCOPO: a atividade pede apenas o saque (valor, validações e
 * distribuição das notas). Os 3 clientes com senha, o menu com saldo, depósito e
 * extrato e o estoque fixo de notas são acréscimos feitos por iniciativa própria.
 *
 * Os dados dos clientes ficam em vetores de 3 posições:
 *   posição 0 = cliente 1, posição 1 = cliente 2, posição 2 = cliente 3
 */

int main() {
    // ---------- dados dos clientes ----------
    int senha[3] = {1234, 2222, 3333};
    int saldo[3] = {500, 1000, 300};
    int tentativas[3] = {0, 0, 0};
```

```
    bool bloqueado[3] = {false, false, false};
    bool finalizou[3] = {false, false, false};
    int historico[3][10];            // depósito = valor positivo, saque = valor negativo
    int qtdTransacoes[3] = {0, 0, 0};

    // ---------- notas que existem no caixa (quantidade fixa no início) ----------
    int estoque100 = 10, estoque50 = 10, estoque20 = 20, estoque10 = 20, estoque5 = 20;

    // ---------- contadores: quantas notas de cada tipo serão entregues no saque ----------
    int notas100, notas50, notas20, notas10, notas5;

    int c = 0;                       // cliente atual (0, 1 ou 2)
    int opcao cliente = 0, opcao = 0;
    int senhaDigitada = 0;
    int valorDeposito = 0, valorSaque = 0, restante = 0;
    int totalNotas = 0, i = 0;
    bool logado = false, sair = false, todosFinalizados = false;

    printf("=== Caixa eletronico ===\n");

    // ---------- LOOP CENTRAL ----------
    // termina quando os 3 clientes saírem (ou forem bloqueados) ou quando as notas acabarem
    while (true) {
        totalNotas = estoque100 + estoque50 + estoque20 + estoque10 + estoque5;

        todosFinalizados = true;
        for (i = 0; i < 3; i++) {
            if (finalizou[i] == false && bloqueado[i] == false) {
                todosFinalizados = false;
            }
        }

        if (totalNotas == 0) {
            printf("\nO caixa ficou sem notas. Encerrando.\n");
            break;
        }
        if (todosFinalizados == true) {
            printf("\nTodos os clientes ja foram atendidos. Encerrando.\n");
            break;
        }

        // ---------- escolha do cliente ----------
        printf("\nInforme qual cliente e:\n");
        printf("1 - Cliente 1\n");
        printf("2 - Cliente 2\n");
        printf("3 - Cliente 3\n");
        scanf("%d", &opcao_cliente);

        if (opcao_cliente < 1 || opcao_cliente > 3) {
            printf("Cliente invalido!\n");
            continue;
        }
        c = opcao cliente - 1;       // 1 vira 0, 2 vira 1, 3 vira 2

        if (finalizou[c] == true) {
            printf("Este cliente ja finalizou o atendimento.\n");
            continue;
        }
        if (bloqueado[c] == true) {
            printf("Cartao bloqueado!\n");
            continue;
        }

        // ---------- pede a senha (3 tentativas) ----------
        logado = false;
        while (tentativas[c] < 3 && logado == false) {
            printf("Digite sua senha:\n");
            scanf("%d", &senhaDigitada);

            if (senhaDigitada == senha[c]) {
                logado = true;
                tentativas[c] = 0;
            } else {
                tentativas[c] = tentativas[c] + 1;
                if (tentativas[c] < 3) {
                    printf("Senha incorreta! Tentativas restantes: %d\n", 3 - tentativas[c]);
```

```
            }
        }
    }

    // se errou 3 vezes o cartão é bloqueado
    if (logado == false) {
        bloqueado[c] = true;
        printf("Senha errada 3 vezes. Cartao bloqueado!\n");
        continue;
    }

    printf("Bem-vindo, cliente %d!\n", opcao_cliente);

    // ---------- menu principal (repete até o cliente escolher sair) ----------
    sair = false;
    while (sair == false) {
        printf("\nEscolha uma das seguintes opcoes:\n");
        printf("1 - Ver saldo\n");
        printf("2 - Depositar\n");
        printf("3 - Sacar\n");
        printf("4 - Extrato\n");
        printf("0 - Sair\n");
        scanf("%d", &opcao);

        switch (opcao) {
            case 1:
                printf("Seu saldo e: R$ %d\n", saldo[c]);
                break;

            case 2:
                printf("Digite o valor do deposito (multiplo de 5):\n");
                scanf("%d", &valorDeposito);

                if (valorDeposito <= 0) {
                    printf("Digite um valor maior que zero.\n");
                } else if (valorDeposito % 5 != 0) {
                    printf("So aceitamos notas de 100, 50, 20, 10 e 5. Digite um multiplo de 5.\n");
                } else {
                    saldo[c] = saldo[c] + valorDeposito;

                    // o extrato guarda as últimas 10 transações: se já estiver cheio,
                    // descarta a mais antiga (posição 0) e desloca as outras uma posição
                    if (qtdTransacoes[c] == 10) {
                        for (i = 0; i < 9; i++) {
                            historico[c][i] = historico[c][i + 1];
                        }
                        qtdTransacoes[c] = 9;
                    }
                    historico[c][qtdTransacoes[c]] = valorDeposito;
                    qtdTransacoes[c] = qtdTransacoes[c] + 1;

                    printf("Deposito realizado! Novo saldo: R$ %d\n", saldo[c]);
                }
                break;

            case 3:
                // ----- 1) Solicita o valor do saque -----
                printf("Digite o valor do saque: ");
                scanf("%d", &valorSaque);

                // ----- 2) Valida o valor: positivo, múltiplo de 5 e dentro do saldo -----
                if (valorSaque <= 0) {
                    printf("Valor invalido! Digite um valor maior que zero.\n");
                } else if (valorSaque % 5 != 0) {
                    // % devolve o resto da divisão: se não for 0, não é múltiplo de 5
                    printf("Valor invalido! O saque deve ser multiplo de 5 (notas de 100, 50, 20, 10
e 5).\n");
                } else if (valorSaque > saldo[c]) {
                    printf("Saldo insuficiente! Seu saldo e R$ %d.\n", saldo[c]);
                } else {
                    // ----- 3) Calcula as notas (sempre começando pela maior) -----
                    // "/" (divisão inteira) diz quantas notas cabem no valor.
                    // "%" (módulo) diz quanto sobra depois dessas notas.
                    // Exemplo: 285 / 100 = 2 notas e 285 % 100 = 85 de restante.
                    // Se o caixa tiver menos notas do que o necessário, entrega só as que
                    // existem e o que não foi pago continua no restante para as notas menores.
```

```
                        restante = valorSaque;

                        // notas de 100
                        notas100 = restante / 100;
                        if (notas100 > estoque100) {
                            notas100 = estoque100;
                            restante = restante - notas100 * 100;
                        } else {
                            restante = restante % 100;
                        }

                        // notas de 50
                        notas50 = restante / 50;
                        if (notas50 > estoque50) {
                            notas50 = estoque50;
                            restante = restante - notas50 * 50;
                        } else {
                            restante = restante % 50;
                        }

                        // notas de 20
                        notas20 = restante / 20;
                        if (notas20 > estoque20) {
                            notas20 = estoque20;
                            restante = restante - notas20 * 20;
                        } else {
                            restante = restante % 20;
                        }

                        // notas de 10
                        notas10 = restante / 10;
                        if (notas10 > estoque10) {
                            notas10 = estoque10;
                            restante = restante - notas10 * 10;
                        } else {
                            restante = restante % 10;
                        }

                        // notas de 5
                        notas5 = restante / 5;
                        if (notas5 > estoque5) {
                            notas5 = estoque5;
                            restante = restante - notas5 * 5;
                        } else {
                            restante = restante % 5;
                        }

                        // ----- 4) Mostra o resultado ou a mensagem de erro -----
                        if (restante != 0) {
                            // sobrou valor sem nota para pagar: o caixa não consegue entregar
                            printf("O caixa nao tem notas suficientes para esse valor. Tente outro
    valor.\n");
                        } else {
                            // tira as notas do caixa e o dinheiro da conta
                            estoque100 = estoque100 - notas100;
                            estoque50 = estoque50 - notas50;
                            estoque20 = estoque20 - notas20;
                            estoque10 = estoque10 - notas10;
                            estoque5 = estoque5 - notas5;
                            saldo[c] = saldo[c] - valorSaque;

                            // o extrato guarda as últimas 10 transações: se já estiver cheio,
                            // descarta a mais antiga (posição 0) e desloca as outras uma posição
                            if (qtdTransacoes[c] == 10) {
                                for (i = 0; i < 9; i++) {
                                    historico[c][i] = historico[c][i + 1];
                                }
                                qtdTransacoes[c] = 9;
                            }
                            historico[c][qtdTransacoes[c]] = -valorSaque;
                            qtdTransacoes[c] = qtdTransacoes[c] + 1;

                            // só mostra as notas com quantidade maior que zero
                            printf("\nNotas entregues:\n");
                            if (notas100 > 0) printf("%d nota(s) de 100\n", notas100);
                            if (notas50 > 0)  printf("%d nota(s) de 50\n", notas50);
```

```
                            if (notas20 > 0)  printf("%d nota(s) de 20\n", notas20);
                            if (notas10 > 0)  printf("%d nota(s) de 10\n", notas10);
                            if (notas5 > 0)   printf("%d nota(s) de 5\n", notas5);

                            printf("\nNovo saldo: R$ %d\n", saldo[c]);

                            // se o caixa zerou, só avisa (o cliente continua no menu)
                            totalNotas = estoque100 + estoque50 + estoque20 + estoque10 + estoque5;
                            if (totalNotas == 0) {
                                printf("\nO caixa ficou sem notas! Voce ainda pode ver saldo e
      extrato.\n");
                            }
                        }
                    }
                    break;

                case 4:   // Extrato
                    if (qtdTransacoes[c] == 0) {
                        printf("Nenhuma transacao realizada.\n");
                    } else {
                        printf("\n--- Extrato ---\n");
                        for (i = 0; i < qtdTransacoes[c]; i++) {
                            if (historico[c][i] > 0) {
                                printf("%d) Deposito: R$ %d\n", i + 1, historico[c][i]);
                            } else {
                                printf("%d) Saque:    R$ %d\n", i + 1, -historico[c][i]);
                            }
                        }
                        printf("Obs.: o extrato mostra apenas as ultimas 10 transacoes.\n");
                    }
                    break;

                case 0: //sair
                    printf("Obrigado e ate logo, cliente %d!\n", opcao_cliente);
                    sair = true;
                    break;

                default:
                    printf("Opcao invalida!\n");
            }
        }

        finalizou[c] = true;
    }

    return 0;
}
```

PARTE 2

# Atividade 02 – Caixa Eletrônico – Grupo 2

*Pseudocódigo, implementação em C e validação por testes*

RELATÓRIO DE DESENVOLVIMENTO

Algoritmos e Pensamento Computacional · Integrantes:
Gabriel Anjos Albuquerque
João Pedro Theodoro de Sousa Andrade
Samuel Carmoni de Oliveira
Petterson Michel Lima Aguilar

## 1. Contextualização e objetivo

Esta atividade propõe a construção de um caixa eletrônico simplificado, implementado em linguagem C, para receber uma solicitação de saque, validar as regras da operação e indicar a quantidade de cédulas a entregar. A solução utiliza estruturas condicionais, divisão inteira e operador módulo.

> **OBJETIVO DA SOLUÇÃO**
> Processar um saque inteiro com saldo disponível de R$ 500, aceitando apenas valores positivos, múltiplos de R$ 5 e não superiores ao saldo. Para saques válidos, decompor o valor em notas de R$ 100, R$ 50, R$ 20, R$ 10 e R$ 5.

## 2. Entradas, saídas e validações

| Elemento | Descrição |
|---|---|
| **Entrada** | Valor inteiro solicitado para saque (valorSaque). |
| **Dado inicial** | Saldo fixo disponível de R$ 500 (saldo). |
| **Saída válida** | Quantidade de notas de cada denominação utilizada; mostra somente quantidades maiores que zero. |
| **Saída inválida** | Mensagem de erro específica: valor não positivo, não múltiplo de 5 ou saldo insuficiente. |

A ordem de validação é importante: primeiro se verifica se o valor é positivo; em seguida, se é múltiplo de 5; por último, se não ultrapassa R$ 500. Somente após essas verificações ocorre a distribuição de notas.

## 3. Identificação das variáveis

| Variável | Tipo | Finalidade |
|---|---|---|
| `valorSaque` | int | Valor solicitado pelo usuário. |
| `saldo` | int | Saldo disponível na operação; inicializado em 500. |
| `restante` | int | Parte do valor ainda não distribuída em notas. |
| `notas100` | int | Contador de notas de R$ 100. |
| `notas50` | int | Contador de notas de R$ 50. |
| `notas20` | int | Contador de notas de R$ 20. |
| `notas10` | int | Contador de notas de R$ 10. |
| `notas5` | int | Contador de notas de R$ 5. |

Todas as variáveis são inteiras, pois a atividade trabalha com valores de saque sem centavos e contagens de cédulas.

## 4. Pseudocódigo

O pseudocódigo abaixo corresponde à solução desenvolvida, preservando a sequência de validação e distribuição de notas que será implementada em C.

**Listagem 1 — Pseudocódigo do caixa eletrônico**

```
ALGORITMO "Pseudocode"
VAR
 valorSaque, saldo, restante, notas100, notas50, notas20, notas10, notas5: INTEIRO
INICIO
   saldo <- 500

   ESCREVA("Digite o valor do saque: ")
   LEIA(valorSaque)

   SE valorSaque <= 0 ENTAO
      ESCREVAL("Erro: Saldo incorreto")
   SENAO
      SE valorSaque MOD 5 <> 0 ENTAO
         ESCREVAL("Erro: valor deve ser multiplo de 5.")
      SENAO
         SE valorSaque > saldo ENTAO
            ESCREVAL("Erro: saldo insuficiente.")
         SENAO
            notas100 <- valorSaque DIV 100
            restante <- valorSaque MOD 100

            notas50 <- restante DIV 50
            restante <- restante MOD 50

            notas20 <- restante DIV 20
            restante <- restante MOD 20

            notas10 <- restante DIV 10
            restante <- restante MOD 10

            notas5 <- restante DIV 5
            restante <- restante MOD 5

            ESCREVAL("Notas entregues:")
            SE notas100 > 0 ENTAO
               ESCREVAL(notas100, " nota(s) de 100")
            FIMSE
            SE notas50 > 0 ENTAO
```

```
               ESCREVAL(notas50, " nota(s) de 50")
            FIMSE
            SE notas20 > 0 ENTAO
               ESCREVAL(notas20, " nota(s) de 20")
            FIMSE
            SE notas10 > 0 ENTAO
               ESCREVAL(notas10, " nota(s) de 10")
            FIMSE
            SE notas5 > 0 ENTAO
               ESCREVAL(notas5, " nota(s) de 5")
            FIMSE
         FIMSE
      FIMSE
   FIMSE
FIMALGORITMO
```

## 5. Implementação em linguagem C

A seguir, apresenta-se o código-fonte fornecido para a atividade, com os comentários inline e as mesmas regras do pseudocódigo. A listagem foi dividida visualmente em duas partes, mas constitui um único programa.

**Listagem 2 — Código em C (parte 1 de 2)**

```c
//Grupo - 2
#include <stdio.h>
int main() {
    int valorSaque, saldo, restante;
    int notas100, notas50, notas20, notas10, notas5;

    //define o saldo inicial da conta
    saldo = 500;

    //lê o valor do saque
    printf("Digite o valor do saque: ");
    scanf("%d", &valorSaque);

    //o valor deve ser positivo
    if (valorSaque <= 0) {
        printf("Erro: Saldo incorreto\n");
    } else {
        // o valor tem que ser multiplo de 5
        if (valorSaque % 5 != 0) {
            printf("Erro: valor deve ser multiplo de 5.\n");
        } else {
            // o valor nao pode ser maior que o saldo
            if (valorSaque > saldo) {
                printf("Erro: saldo insuficiente.\n");
            } else {
                // calculo da quantidade de notas, começando pela maior

                // Notas de 100: quantas cabem no valor e o que sobra
                notas100 = valorSaque / 100;
                restante = valorSaque % 100;

                // Notas de 50: quantas cabem no restante e o que sobra
                notas50 = restante / 50;
                restante = restante % 50;
```

**Listagem 2 — Código em C (continuação, parte 2 de 2)**

```
                // Notas de 20: quantas cabem no restante e o que sobra
                notas20 = restante / 20;
                restante = restante % 20;

                // Notas de 10: quantas cabem no restante e o que sobra
                notas10 = restante / 10;
                restante = restante % 10;

                // Notas de 5: quantas cabem no restante e o que sobra
                notas5 = restante / 5;
                restante = restante % 5;

                // Exibe o resultado
                printf("Notas entregues:\n");
                if (notas100 > 0) {
                    printf("%d nota(s) de 100\n", notas100);
                }
                if (notas50 > 0) {
                    printf("%d nota(s) de 50\n", notas50);
                }
                if (notas20 > 0) {
                    printf("%d nota(s) de 20\n", notas20);
                }
                if (notas10 > 0) {
                    printf("%d nota(s) de 10\n", notas10);
                }
                if (notas5 > 0) {
                    printf("%d nota(s) de 5\n", notas5);
                }
            }
        }
    }

    return 0;
}
```

### 5.1. Funcionamento do algoritmo

O programa recebe o valor pelo scanf e executa verificações com if/else. Para um saque válido, começa pela cédula de maior valor e prossegue até R$ 5. Cada divisão inteira calcula o número de cédulas; cada operação de módulo atualiza o restante. Ao final, a saída é condicionada a contadores superiores a zero.

**Exemplo: no saque de R$ 285, a decomposição é 2 × 100 + 1 × 50 + 1 × 20 + 1 × 10 + 1 × 5 = R$ 285.**

## 6. Documentação de testes

Foram executados nove cenários, incluindo os quatro valores explicitamente sugeridos no enunciado. Todos os resultados abaixo foram obtidos pela execução do programa.

## 6.1. Resumo das execuções

| Valor | Cenário | Resultado observado | Situação |
|---|---|---|---|
| **R$ 50** | Válido | 1 nota de R$ 50 | **Correto** |
| **R$ 95** | Válido | 1 × 50 + 2 × 20 + 1 × 5 | **Correto** |
| **R$ 123** | Inválido | Valor não múltiplo de 5 | **Correto** |
| **R$ 1.005** | Inválido | Saldo insuficiente | **Correto** |
| **R$ 0** | Inválido | Valor não positivo | **Correto** |
| **R$ -10** | Inválido | Valor não positivo | **Correto** |
| **R$ 285** | Válido | 2 × 100 + 50 + 20 + 10 + 5 | **Correto** |
| **R$ 500** | Válido | 5 notas de R$ 100 | **Correto** |
| **R$ 5** | Válido | 1 nota de R$ 5 | **Correto** |

Critério de aprovação: em valores válidos, as notas somam exatamente o valor informado; em valores inválidos, nenhuma nota é entregue e é exibida a mensagem de erro adequada.

## 6.2. Entradas, saídas reais e análises

### Teste 1 — saque de R$ 50

**Classificação:** Válido — múltiplo de 5 | Saldo inicial: R$ 500

```
Digite o valor do saque: 50
Notas entregues:
1 nota(s) de 50
```

**Análise:** A solicitação é correta! O valor é divisível por 5 e menor que o saldo. O programa entrega 1 nota de R$ 50, totalizando exatamente R$ 50.

### Teste 2 — saque de R$ 95

**Classificação:** Válido — múltiplo de 5 | Saldo inicial: R$ 500

```
Digite o valor do saque: 95
Notas entregues:
1 nota(s) de 50
2 nota(s) de 20
1 nota(s) de 5
```

**Análise:** A solicitação é correta! O valor é aceito e distribuído em 1 nota de R$ 50, 2 de R$ 20 e 1 de R$ 5. A soma (50 + 40 + 5) corresponde a R$ 95.

### Teste 3 — saque de R$ 123

**Classificação:** Inválido — não múltiplo de 5 | Saldo inicial: R$ 500

```
Digite o valor do saque: 123
Erro: valor deve ser multiplo de 5.
```

**Análise:** A solicitação é correta! O programa rejeita o saque antes de calcular as notas, exibindo o erro previsto.

### Teste 4 — saque de R$ 1005

**Classificação:** Inválido — saldo insuficiente | Saldo inicial: R$ 500

```
Digite o valor do saque: 1005
Erro: saldo insuficiente.
```

**Análise:** A solicitação é correta! Embora o valor seja positivo e múltiplo de 5, R$ 1.005 ultrapassa os R$ 500 disponíveis; por isso, é exibida a mensagem de saldo insuficiente.

### Teste 5 — saque de R$ 0

**Classificação:** Inválido — valor zero | Saldo inicial: R$ 500

```
Digite o valor do saque: 0
Erro: Saldo incorreto
```

**Análise:** A solicitação é correta! O saque é rejeitado pela primeira validação, que exige valor maior que zero. Nenhuma nota é calculada.

### Teste 6 — saque de R$ -10

**Classificação:** Inválido — valor negativo | Saldo inicial: R$ 500

```
Digite o valor do saque: -10
Erro: Saldo incorreto
```

**Análise:** A solicitação é correta! O saque negativo é bloqueado pela condição valorSaque <= 0. A resposta é a mesma aplicada ao valor zero.

### Teste 7 — saque de R$ 285

**Classificação:** Válido — exemplo do enunciado | Saldo inicial: R$ 500

```
Digite o valor do saque: 285
Notas entregues:
2 nota(s) de 100
1 nota(s) de 50
1 nota(s) de 20
1 nota(s) de 10
1 nota(s) de 5
```

**Análise:** A solicitação é correta! O programa entrega 2 notas de R$ 100, 1 de R$ 50, 1 de R$ 20, 1 de R$ 10 e 1 de R$ 5. A soma confirma R$ 285.

### Teste 8 — saque de R$ 500

**Classificação:** Válido — limite do saldo | Saldo inicial: R$ 500

```
Digite o valor do saque: 500
Notas entregues:
5 nota(s) de 100
```

**Análise:** O saque é igual ao saldo disponível, portanto é permitido. São entregues 5 notas de R$ 100, somando R$ 500.

### Teste 9 — saque de R$ 5

**Classificação:** Válido — menor denominação | Saldo inicial: R$ 500

```
Digite o valor do saque: 5
Notas entregues:
1 nota(s) de 5
```

**Análise:** A quantia é positiva, múltipla de 5 e está dentro do saldo. O programa exibe apenas 1 nota de R$ 5.

## 7. Avaliação dos resultados

As nove execuções apresentaram o comportamento esperado. Os cenários válidos demonstraram a distribuição de notas em ordem decrescente, com exibição apenas de denominações utilizadas. Os cenários inválidos demonstraram que as verificações de sinal, divisibilidade e saldo impedem o processamento de saques não permitidos.

PARTE 3

# Feedback entre os grupos

Os grupos trocaram os códigos em C e cada um fez um feedback sobre o código do outro. Abaixo estão os dois feedbacks.

## Feedback do código do Grupo 1 para o Grupo 2

*Elaborado pelo Grupo 1 a partir do código caixa_eletronico.c (Grupo 2).*

O código do Grupo 2 está correto e atende ao que a atividade pede: lê o valor do saque, válida na ordem certa (maior que zero, múltiplo de 5 e dentro do saldo) e distribui as notas começando pela de 100, usando a divisão inteira (/) e o resto da divisão (%). Também mostra apenas as notas com quantidade maior que zero. Executamos o programa com os nove valores do relatório (50, 95, 123, 1005, 0, -10, 285, 500 e 5) e todas as saídas foram iguais às documentadas. O código é curto, bem comentado e fácil de entender, e o relatório traz pseudocódigo, tabela de variáveis e testes, inclusive com os casos-limite de R$ 500 e R$ 5.

A principal diferença entre os códigos está no escopo. O Grupo 2 fez somente o saque, que é o que o enunciado pede, enquanto o nosso código acrescentou, por iniciativa própria, 3 clientes com senha, menu com saldo, depósito e extrato, e um estoque de notas no caixa. Por isso, no código do Grupo 2 o saldo fixo de R$ 500 não é atualizado depois do saque e o caixa não controla quantas notas tem, o que não é um erro, já que isso não faz parte do enunciado.

Como sugestão, a mensagem “Erro: Saldo incorreto”, mostrada quando o valor é 0 ou negativo, poderia ser algo como “Erro: o valor deve ser maior que zero”, pois o problema está no valor digitado e não no saldo. No geral, o trabalho está bem-feito.

**Resumo das principais diferenças**

| Aspecto | Grupo 1 | Grupo 2 |
|---|---|---|
| **Escopo** | Saque, 3 clientes com senha, menu com saldo, depósito e extrato | Somente o saque, como pede o enunciado |
| **Saldo** | Um saldo por cliente (R$ 500, R$ 1000 e R$ 300), atualizado a cada saque ou depósito | Fixo em R$ 500, sem atualização depois do saque |
| **Notas no caixa** | Estoque limitado (10 notas de 100, 10 de 50 e 20 de cada uma das notas de 20, 10 e 5), que diminui a cada saque | Sem estoque: entrega sempre as notas calculadas |
| **Validações do saque** | Maior que zero, múltiplo de 5, dentro do saldo e notas suficientes no caixa | Maior que zero, múltiplo de 5 e dentro do saldo |
| **Mensagem para valor 0 ou negativo** | “Valor invalido! Digite um valor maior que zero.” | “Erro: Saldo incorreto” |
| **Estrutura do código** | Laço while, vetores e switch | if/else aninhados e variáveis simples |

# Feedback do código do Grupo 2 para o Grupo 1

*Elaborado pelo Grupo 2 a partir do código caixa_eletronico_atividade2.c (Grupo 1).*

O código está bem estruturado e demonstrou maior diferença em implementar funcionalidades adicionais, como autenticação por senha, bloqueio após três tentativas incorretas, depósito, consulta de saldo e extrato de transações. Os comentários ao longo do código facilitaram em muito a compreensão da lógica, e o uso de vetores permite organizar os dados dos diferentes clientes.

Logo assim, o trabalho demonstra boa organização, uso adequado de estruturas condicionais e operadores aritméticos, além de uma iniciativa interessante de ir muito a mais do que foi proposto pela atividade, trazendo um diferencial para o código. Com a documentação dos testes, a inclusão do pseudocódigo e alguns outros detalhes da atividade, o projeto poderá atender até a mais do que é exigido a atividade, o tornando bem completo!